

LV07 재귀함수 탐색 순서

■ 소유자	 암암코딩
■ 태그	
■ 생성 일시	@2024년 1월 13일 오후 6:35

<https://youtu.be/6RcR56GpFi8?feature=shared>

https://youtu.be/4Lye_XHhRco?feature=shared

C++ 재귀 경로 추적과 다차원 배열

재귀함수 경로 추적 (Path Tracking)

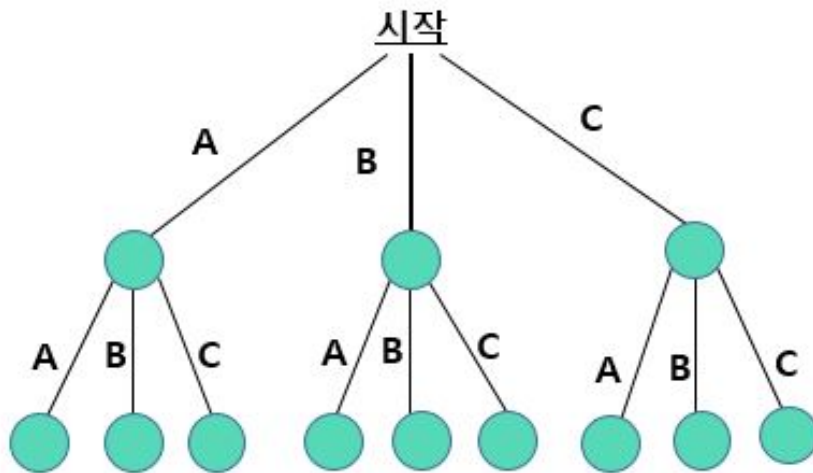
재귀 경로 추적의 개념

재귀함수를 탐색하다 보면 딥 그림과 같이 무수히 많은 함수를 어떤 함수를 실행하고 있는지 따로 체크를 하지 않으면 탐색하기가 어렵다.

특히 디버깅을 하거나 할때 헛갈리기 쉽상이다.

그래서 함수에 들어가기 전에 해당 함수의 위치를 배열에 기록해두면 됩니다.

트리 구조와 경로 추적



기록해두기 위한 배열을 만들어 두고 한단계 밑으로 레벨이 증가할때마다 배열의 인덱스(레벨)로 해당 들어가는 위치 또는 방향을 기록해두면 됩니다.

경로 추적 구현

```

#include <iostream>
using namespace std;

char path[10] = "";
char name[10] = "ABC";

void run(int level)
{
    if (level == 2)
    {
        cout << path << endl;
        cout << endl;
        return;
    }

    //path[level] = 'A';
    //run(level + 1);
    //path[level] = 0;
    //path[level] = 'B';
  
```

```

    //run(level + 1);
    //path[level] = 0;

    for (size_t i = 0; i < 3; i++)
    {
        path[level] = name[i];
        run(level + 1);
        path[level] = 0;
    }
}

int main()
{
    run(0);

    return 0;
}

```

코드 동작 원리:

1. **path 배열**: 현재까지의 경로를 저장
2. **level 매개변수**: 현재 재귀 깊이를 나타냄
3. **경로 기록**: `path[level] = name[i]` 로 현재 선택 저장
4. **재귀 호출**: 다음 레벨로 진행
5. **백트래킹**: `path[level] = 0` 으로 경로 초기화

실행 결과:

```

AA
AB
AC
BA
BB
BC
CA
CB
CC

```

경로 추적의 상세 과정

레벨 0에서의 실행:

1. `path[0] = 'A'` , `run(1)` 호출
2. `path[0] = 'B'` , `run(1)` 호출
3. `path[0] = 'C'` , `run(1)` 호출

레벨 1에서의 실행 (`path[0] = 'A'`인 경우):

1. `path[1] = 'A'` , `run(2)` 호출 → "AA" 출력
2. `path[1] = 'B'` , `run(2)` 호출 → "AB" 출력
3. `path[1] = 'C'` , `run(2)` 호출 → "AC" 출력

백트래킹 (Backtracking) 이해

```
void runWithDebug(int level)
{
    cout << "Level " << level << " 진입, 현재 path: " << path << endl;

    if (level == 2)
    {
        cout << "최종 경로: " << path << endl;
        return;
    }

    for (size_t i = 0; i < 3; i++)
    {
        cout << "Level " << level << "에서 " << name[i] << " 선택" << endl;
        path[level] = name[i];

        runWithDebug(level + 1);

        cout << "Level " << level << "로 백트래킹, " << name[i] << " 제거" << endl;
        path[level] = 0;
    }
}
```

백트래킹의 핵심:

- **선택:** 현재 레벨에서 가능한 선택을 함
- **탐색:** 선택한 상태로 다음 레벨 탐색
- **복구:** 탐색이 끝나면 선택을 취소하고 원상복구
- **반복:** 다른 선택지들에 대해 같은 과정 반복

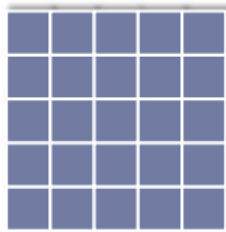
다차원 배열 (Multi-dimensional Array)



1차원 배열 arrOneDim



3차원 배열 arrThreeDim



2차원 배열 arrTwoDim

다차원 배열의 개념

다차원 배열이란 2차원 이상의 배열을 의미하며, 배열 요소로 또 다른 배열을 가지는 배열을 의미합니다.

차원별 정의:

- **2차원 배열:** 배열 요소로 1차원 배열을 가지는 배열
- **3차원 배열:** 배열 요소로 2차원 배열을 가지는 배열
- **4차원 배열:** 배열 요소로 3차원 배열을 가지는 배열

다차원 배열 시각화

1차원 배열 arrOneDim

[□][□][□][□][□][□][□][□][□][□]

2차원 배열 arrTwoDim

[□][□][□][□][□]

[□][□][□][□][□]

[□][□][□][□][□]

[][][][][]

3차원 배열 arrThreeDim

여러 개의 2차원 배열이 층층이 쌓인 형태

다차원 배열 선언과 초기화

```
#include <iostream>
using namespace std;

int main()
{
    // 1차원 배열
    int arr1D[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // 2차원 배열
    int arr2D[3][4] =
    {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // 3차원 배열
    int arr3D[2][3][4] =
    {
        {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12}
        },
        {
            {13, 14, 15, 16},
            {17, 18, 19, 20},
            {21, 22, 23, 24}
        }
    };
};
```

```

// 4차원 배열
int arr4D[2][2][3][4];

return 0;
}

```

다차원 배열 접근

```

#include <iostream>
using namespace std;

int main()
{
    int arr3D[2][3][4] =
    {
        {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12}
        },
        {
            {13, 14, 15, 16},
            {17, 18, 19, 20},
            {21, 22, 23, 24}
        }
    };

    // 3차원 배열 접근
    cout << "arr3D[0][1][2] = " << arr3D[0][1][2] << endl; // 7
    cout << "arr3D[1][2][3] = " << arr3D[1][2][3] << endl; // 24

    // 3중 반복문으로 전체 출력
    for (int i = 0; i < 2; i++)    // 깊이
    {
        cout << "Layer " << i << ":" << endl;
        for (int j = 0; j < 3; j++) // 행
        {
            for (int k = 0; k < 4; k++) // 열

```

```

    {
        cout << arr3D[i][j][k] << " ";
    }
    cout << endl;
}
cout << endl;
}

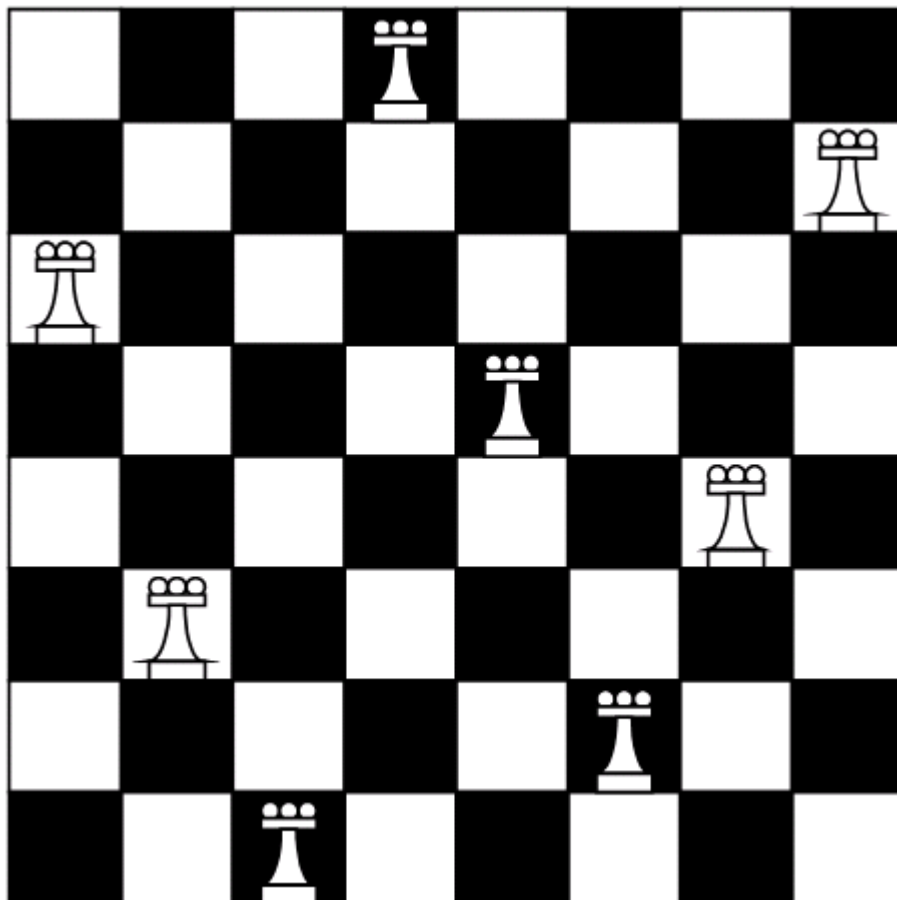
return 0;
}

```

재귀와 다차원 배열 활용

N-Queen 문제

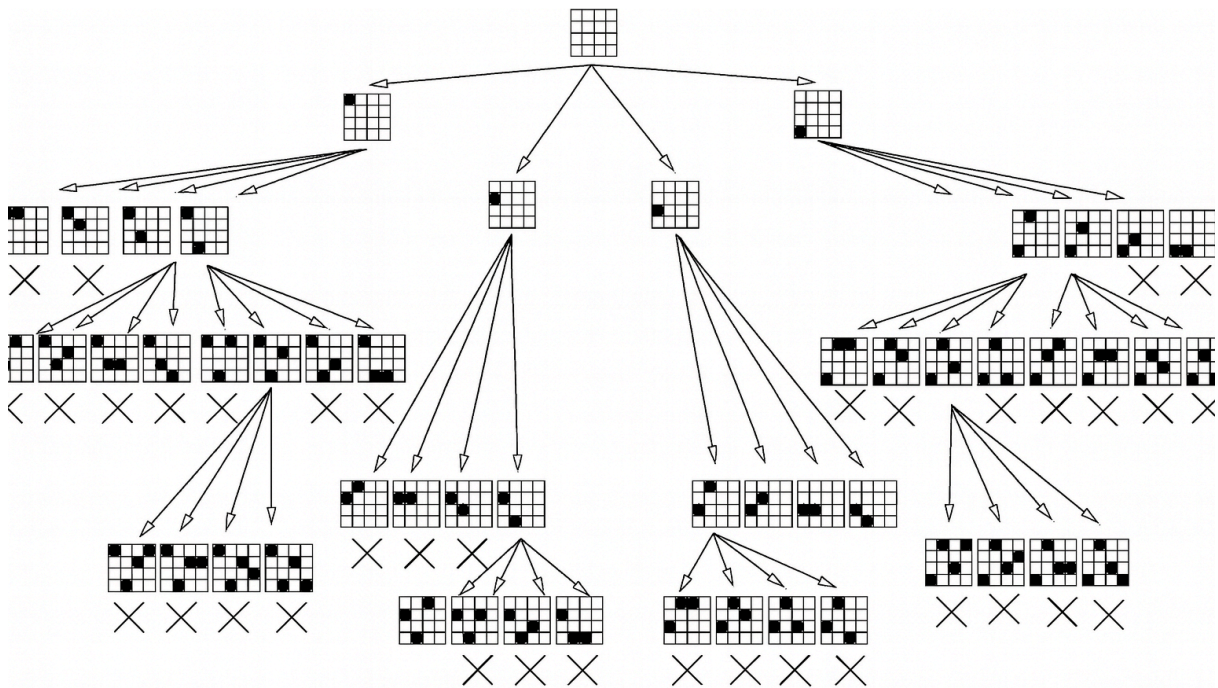
N-Queen 퍼즐은 $N \times N$ 크기의 체스판에 N 개의 퀸을, 서로 공격할 수 없도록 올려놓는 퍼즐입니다. (퀸은 체스에서 가장 강력한 기물로, 자신의 위치에서 상하좌우, 그리고 대각선 방향으로 이어진 직선 상의 어떤 기물도 공격할 수 있습니다)



자 이제 실제로 문제를 풀어보죠! 위처럼 백트래킹알고리즘을 사용하기 위해 유망한지를 검사하는 규칙을 정합니다.

1. 퀸은 같은 열에 있으면 안됩니다. (물론 같은 행에 있어도 안됩니다. 이 조건은 유망성검사를 하기 전에 걸러집니다.)
2. 퀸은 같은 " 방향 대각선에 있으면 안됩니다.
3. 퀸은 같은 "/" 방향 대각선에 있으면 안됩니다.

위와 같은 규칙을 가지고 깊이 우선 탐색을 시작합니다. 그러면 N = 4인 경우 아래와 같은 그림이 될 것입니다.



```
#include <iostream>
using namespace std;

class NQueens
{
public:
    NQueens(int n) : mSize(n)
    {
```

```

    for (int i = 0; i < MAX_SIZE; i++)
    {
        for (int j = 0; j < MAX_SIZE; j++)
        {
            mBoard[i][j] = 0;
        }
        mPath[i] = -1;
    }
}

void Solve()
{
    SolveRecursive(0);
    cout << "총 " << mSolutionCount << "개의 해가 있습니다." << endl;
}

private:
    static const int MAX_SIZE = 10;
    int mBoard[MAX_SIZE][MAX_SIZE];
    int mPath[MAX_SIZE]; // 각 행에서 퀸의 열 위치
    int mSize;
    int mSolutionCount = 0;

    bool IsSafe(int row, int col)
    {
        // 같은 열에 퀸이 있는지 확인
        for (int i = 0; i < row; i++)
        {
            if (mPath[i] == col)
                return false;
        }

        // 대각선에 퀸이 있는지 확인
        for (int i = 0; i < row; i++)
        {
            //행 차이 = abs(i - row)
            //열 차이 = abs(mPath[i] - col)
            //행,열차이가 같으면 대각선에 퀸이 있음

```

```

        if (abs(mPath[i] - col) == abs(i - row))
            return false;
    }

    return true;
}

void SolveRecursive(int row)
{
    if (row == mSize)
    {
        mSolutionCount++;
        PrintSolution();
        return;
    }

    for (int col = 0; col < mSize; col++)
    {
        if (IsSafe(row, col))
        {
            mPath[row] = col;    // 경로 기록
            mBoard[row][col] = 1; // 퀸 배치
            SolveRecursive(row + 1); // 다음 행으로
            mBoard[row][col] = 0; // 백트래킹
            mPath[row] = -1;
        }
    }
}

void PrintSolution()
{
    cout << "Solution " << mSolutionCount << ":" << endl;
    for (int i = 0; i < mSize; i++)
    {
        for (int j = 0; j < mSize; j++)
        {
            cout << (mBoard[i][j] ? "Q " : ". ");
        }
    }
}

```

```

        cout << endl;
    }
    cout << endl;
}
};

int main()
{
    int n;

    cout << "N-Queens 문제 해결기" << endl;
    cout << "체스판의 크기를 입력하세요 (1-10): ";
    cin >> n;

    // 입력 검증
    if (n < 1 || n > 10)
    {
        cout << "잘못된 입력입니다. 1~10 사이의 숫자를 입력해주세요." << endl;
        return 1;
    }

    cout << endl << n << "x" << n << " 체스판에서 " << n << "개의 퀸을 배치하
는 모든 경우:" << endl << endl;

    NQueens nqueens(n);
    nqueens.Solve();

    return 0;
}

```

“강의는 많은데, 왜 나는 아직도 코드를 못 짤까?”

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,

- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얍얍코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얍얍코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얍얍코딩 상담방](#)

▼ oopy:hide

연습문제

재귀호출을 하다 보면 어느 시점에 있는지 정확히 알기 힘듭니다.

path전역배열을 이용하면 경로를 알 수 있습니다!

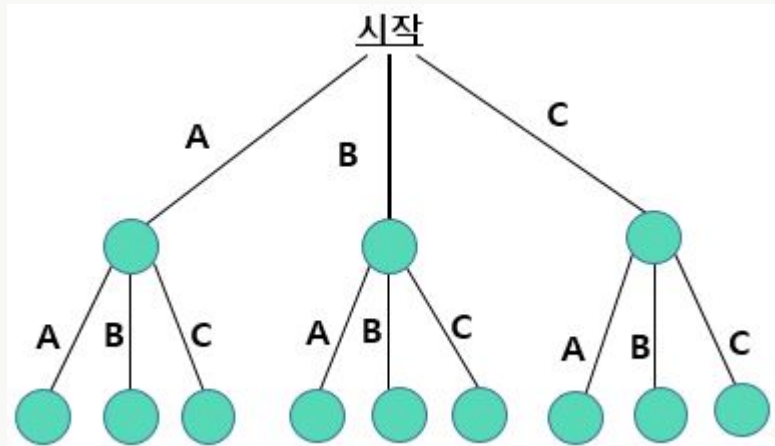
재귀호출에서 필수로 알아야 하는 path를 이해해봅시다.



경로를 출력하자

문제 1번

Level2에 도달했을 때마다, path를 모두 출력 해주세요.



출력 결과

AA

AB

AC

BA

BB

BC

CA

CB

CC



같은문장 찾기

문제 2번

<cstring> strcmp를 사용해서 문제를 풀어주세요.

세문장을 입력 받으세요. (최대글자는 10글자)

세문장이 모두 같으면 **"WOW"**

두문장이 같으면 **"GOOD"**

모두 다르면 **"BAD"** 를 출력 해주세요.

입력 예제

abcd

abcd

abcd

출력 결과

WOW



경로 이름은 BGTK

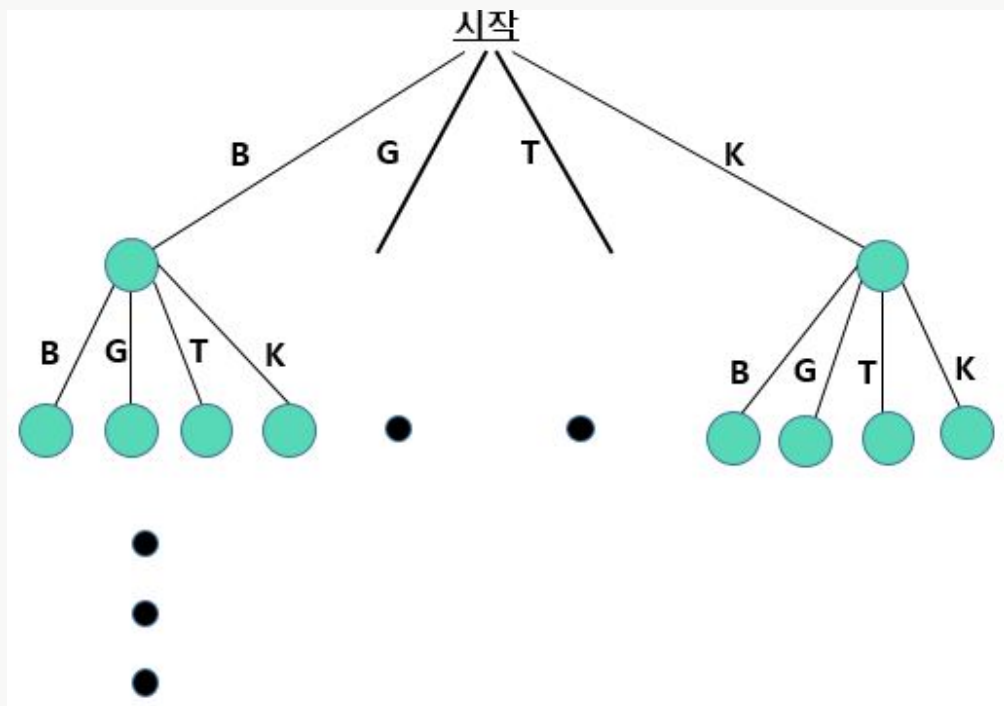
문제 3번

다음과 같은 구조로 **Level**을 입력받고

입력받은 Level에 도달했을 때

path를 출력 하주세요.

(2 <= LEVEL <= 4)



입력 예제

2

출력 결과

BB

BG

BT

BK

GB

GG

GT

GK

TB

TG

TT

TK

KB

KG

KT

KK



Up & Down

문제 4번

up 또는 down 문장을 5개 입력 받으세요.

주인공이 1층에 서있는데

up이 입력으로 들어오면 **2층**으로 가고 (up 입력시 한층 올라감)

down이 들어오면 **B1층**으로 갑니다. (down 입력시 한층 내려감)

up/down 을 **5회 입력** 받았을 때 **주인공의 위치를 출력** 하세요.

ex)

입력:

up

down

down

down

down

출력: B3

입력 예제

up

down

down

down

down

출력 결과

B3



청소당번

문제 5번



1 ~ n 명의 친구가 4일치 청소당번 순서를 정하려고 합니다.

첫번째 친구가 4일치 모두 청소할 수도 있고, 다양한 경우가 발생합니다.

ex) n = 3명일때 가능한 청소스케줄

1111 / 1112 / 1113 / 1121 / 1122 / ... / 3332 / 3333

n명에 대해, 4일 동안의 청소스케줄을 모두 출력하는 프로그램을 작성해주세요.

입력 예제

2

출력 결과

1111

1112

1121

1122

1211

1212

1221

1222

2111

2112

2121

2122

2211

2212

2221

2222



가장 긴문장, 짧은 문장은 어디에

문제 6번

string 1차원 배열에 네문장을 입력 받으세요.

가장 긴 문장과 가장 짧은 문장이 몇번째 index에 있는 문장인지 **index**를 출력 해주세요

ex)

[입력]

apple

microsoft

google

xiaomi

[출력]

긴문장:1 //(microsoft)

짧은문장:0 //(apple)

[hint]

문장의 길이를 구하는 방법

```
string a = "mincoding";
```

```
int len = a.length();
```

입력 예제

apple

microsoft

google

xiaomi

출력 결과

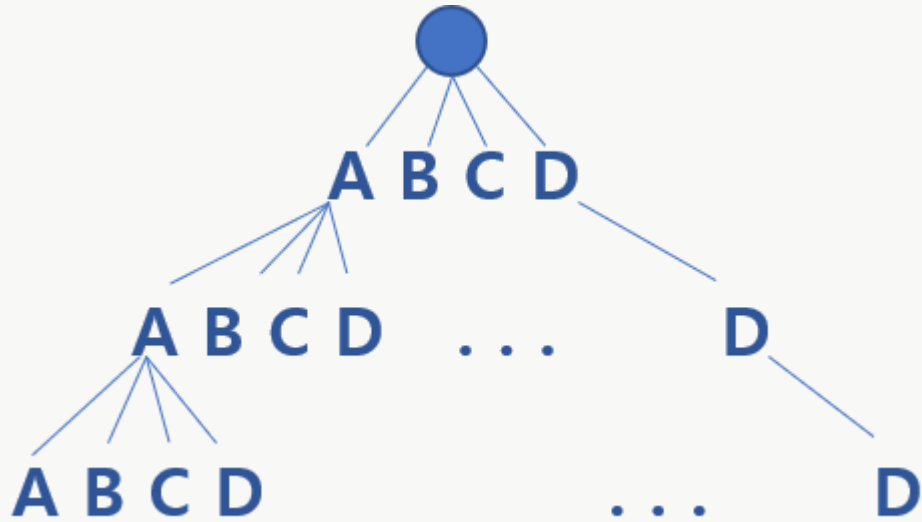
긴문장:1

짧은문장:0



몇번째 행운

문제 7번



위와 같은 트리모양으로 재귀호출 프로그램을 만드려 합니다.

Level 3에 도착했을때 입력 받은 PATH가 몇번째로 만들어지는지 출력 해주
세요.

ex)

AAA ⇒ 1번째

입력 예제

AAD

출력 결과

4번째



3차 배열에서 MAX MIN 찾기

문제 8번

아래와 같이 3차 배열을 하드코딩 해 주세요

2	4	2	3	7	3
1	5	3	6	1	5
[0]		[1]		[2]	

숫자 한개를 입력 받고, 입력받은 위치의 MAX와 MIN값을 출력 해 주세요

ex) 만약 0을 입력받았다면 **[0]에 해당하는 곳에서** MAX값과 MIN값을 출력하면 됩니다

MAX=5

MIN=1

입력 예제

1

출력 결과

MAX=6

MIN=2



암호 해독하기

문제 9번

암호표에 암호문들이 있습니다.

Jason
Dr.tom
EXEXI
GK12P
POW

암호문을 입력 받아주세요.

암호가 맞으면 "암호해제" 라고 출력,

암호가 틀리면 "암호틀림" 라고 출력 해주세요.

입력 예제

abc

출력 결과

암호틀림

복습문제



3차배열과 문자의 발견여부

문제 1번

A	T	B
C	C	B

A	A	A
B	B	C

위 3차배열을 하드코딩 하주세요

그리고 문자 1개를 입력 받아주세요

3차배열에 입력받은 문자가 존재하면 "**발견**", 없다면 "**미발견**"이라고 출력 하주세요

입력 예제

T

출력 결과

발견



바람둥이

문제 2번

나에게 호감을 갖고 있는 n 명의 친구들이 있습니다.

이 중 누구와 데이트할지 **ox**로 데이트 조합을 출력해보려고 합니다.

n 을 입력받고 그에 따른 데이트 조합을 모두 출력 해주세요.

입력 예제

2

출력 결과

XX

XO

OX

OO



문자를 채우다

문제 3번

$3 \times 3 \times 3$ 으로 구성되어 있는 3차 배열을 선언 해 주세요

그리고 문자 하나를 입력 받습니다

아래 예시와 같이 입력 받은 문자부터 순차적으로 값을 채워주세요

ex) 만약 문자 A를 입력하였다면 아래와 같이 채우고 출력

A	A	A	B	B	B	C	C	C
A	A	A	B	B	B	C	C	C
A	A	A	B	B	B	C	C	C

ex) 만약 문자 B를 입력하였다면 아래와 같이 채우고 출력

B	B	B	C	C	C	D	D	D
B	B	B	C	C	C	D	D	D
B	B	B	C	C	C	D	D	D

입력 예제

A

출력 결과

AAA

AAA

AAA

BBB

BBB

BBB

CCC

CCC

CCC



한줄a 한줄b

문제 4번

숫자 2개를 변수 a, b에 입력 받습니다

아래의 규칙에 따라 값을 채워주세요

만약 a에 1, b에 3을 입력 받았다면, 아래와 같이 채우고 출력하면 됩니다.

1	1	1
3	3	3

1	1	1
3	3	3

1	1	1
3	3	3

입력 예제

1 3

출력 결과

1 1 1

3 3 3

1 1 1

3 3 3

1 1 1

3 3 3



문제 5번

문자 2개를 입력 받으세요.

MAP 배열에서 찾을 값을 이용해 **price** 배열의 값을 찾아야 합니다.

예를들어,

만약 **"B3"**을 입력 받았다면 MAP 배열에서 **3**을 찾을 수 있고

price배열의 3번에 해당하는 문자 **G**를 출력 하면 됩니다.

ex) 입력: B 3 , 출력: G

MAP

	1	2	3	4	5	6
A	3	5	4	2	2	3
B	1	3	3	3	4	2
C	5	4	4	2	3	5

price

1	'T'
2	'P'
3	'G'
4	'K'
5	'C'

입력 예제

B 3

출력 결과

G



문장 정렬

문제 6번

길이가 다른 네 문장을 입력 받으세요. (최대글자는 10글자)

가장 짧은 문장부터 긴 문장까지 오름차순 정렬해서 출력 해 주세요.

입력 예제

kfcmclo

zzzzz

abc

mincoding

출력 결과

abc

zzzzz

kfcmclo

mincoding



이니셜 뽑기

문제 7번

3차배열에 다음 값을 하드코딩 하주세요.(ABCD)

	#		#	#	#	#	#	#	#	#		#	#	
#		#	#		#	#		#	#		#	#		#
#	#	#	#	#	#	#			#		#	#		#
#		#	#		#	#		#	#		#	#		#
#		#	#	#	#	#	#	#	#	#	#	#		
[0]			[1]			[2]			[3]					

그리고 숫자 1개를 입력 받으세요.

숫자에 해당하는 배열값을 출력 하주세요.

예로들어 0을 입력했다면

```
#
# #
###
# #
# #
```

와 같이 출력하면 됩니다.

(빈칸 : 띄어쓰기 한칸)

입력 예제

0

출력 결과

```
#
# #
###
# #
```

#